Spring Boot Actuator: A Powerful Toolkit for Application Monitoring

Thurumerla Venkatesh Follow 2 min read · Feb 19, 2024

For developers who rely on Spring Boot, the Actuator library is an invaluable asset. It seamlessly integrates into your applications, offering a wealth of endpoints that expose crucial metrics and health checks, granting you deep visibility into their internal workings. Whether you're tackling a complex microservices architecture or crafting a monolithic behemoth, Actuator empowers you to monitor, manage, and debug with confidence.

Key Monitoring Cases:

1. **Health Checks:**

- **Endpoint:** `/actuator/health`
- **Usage:** Gain immediate insights into your application's overall health, ensuring it's up and running as expected. Identify potential issues or dependencies that might be causing errors early on.
- **Example:**

```
@SpringBootApplication
public class MyApplication {
```

```
    @Bean
    public HealthIndicator databaseHealthIndicator(DataSource dataSource) {
        return new JdbcHealthIndicator(dataSource);
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Metrics

- **Endpoint:** `/actuator/metrics`
- **Usage:** Dive deeper into various performance metrics, such as memory usage, request throughput, and garbage collection activity. Analyze trends and identify potential bottlenecks or resource constraints.
- **Example:**

```
@SpringBootApplication
public class MyApplication {

    @Bean
    public MetricService metricService() {
        MetricServiceImpl metricService = new MetricServiceImpl();
        metricService.addMetric("custom.metric", (Gauge<Long>) () -> yourBusines
        return metricService;
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

3. Environment:

- **Endpoint:** `/actuator/env`
- **Usage:** Inspect the properties loaded into your application's environment, making it easier to debug configuration issues and understand how various settings are being applied.

```
@SpringBootApplication
public class MyApplication {

    @Value("${my.custom.property}")
    private String customProperty;

    @Bean
    public HealthIndicator configurationHealthIndicator() {
        return () -> Health.up().withDetail("customProperty", customProperty);
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

4.Thread Dump:

- **Endpoint:** `/actuator/threads`
- **Usage:** Generate a snapshot of active threads in your application, including their names, states, and stack traces. This is invaluable for troubleshooting deadlocks, performance issues, or identifying hung threads.

```
@SpringBootApplication
public class MyApplication {

    @Bean
    public HealthIndicator threadDumpHealthIndicator() {
        return new ThreadDumpHealthIndicator();
```

```
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

5. HTTP Traces:

- **Endpoint:** `/actuator/httptrace`
- **Usage:** Record traces of HTTP requests and responses, pinpointing issues related to slow-performing requests, routing inefficiencies, or external service delays.
- **Example:**

```
@SpringBootApplication
public class MyApplication extends WebMvcConfigurerAdapter {

    @Override
    public void configureMessageConverters(List<HttpMessageConverter<?>> convert
        TraceHttpMessageConverter traceConverter = new TraceHttpMessageConverter
        converters.add(traceConverter);
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Security Considerations:

- **Authentication:** By default, most Actuator endpoints are insecure. It's crucial to enable appropriate authentication and authorization mechanisms, such as Spring Security, to prevent unauthorized access to sensitive information.
- **Endpoint Exposure:** Carefully evaluate which endpoints you expose via Actuator, restricting access to those that are truly necessary for monitoring and management. Consider leveraging environment variables or a configuration server to manage sensitive endpoint URLs.

Further reference :

https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html

Spring Boot | Spring Framework

Written by Thurumerla Venkatesh

2 followers · 22 following

Follow

No responses yet

More from Thurumerla Venkatesh

Thurumerla Venkatesh

10 Different Ways to Reverse a String in Java (Without using reverse or sort methods)

String manipulation is a common task and reversing strings is one of the fundamental operation. Here, we'll explore ten simple and basic...

Feb 19, 2024

See all from Thurumerla Venkatesh

Recommended from Medium